Subject Name: OBJECT ORIENTED PROGRAMMING DESIGN

Subject Code: CS T33

2 MARKS AND 11 MARKS

UNIT V

Object Modelling and Object Oriented Software development: Overview of OO concepts – UML – Use case model – Class diagrams – Interaction diagrams – Activity diagrams – state chart diagrams - Patterns – Types – Object Oriented Analysis and Design methodology – Interaction Modelling – OOD Goodness criteria.

PART A – 2 MARK QUESTIONS

1. What Is Object Oriented Analysis and Design?

During object-oriented analysis, there is an emphasis on finding and describing the objects—or concepts—in the problem domain. For example, in the case of the library information system, some of the concepts include Book, Library, and Patron.

During object-oriented design, there is an emphasis on defining software objects and how they collaborate to fulfill the requirements. For example, in the library system, a Book software object may have a title attribute and a get Chapter method

2. What are the rules of UML?

The rules of UML are:

Names What you call things, relationships, and diagrams
Scope The context that gives specific meaning to a name
Visibility How those names can be seen and used by others
Integrity How things properly and consistently relate to one Another.
Execution What it means to run or simulate a dynamic model.

3. Define UML(unified modeling language)

UML, as the name implies, is a modeling language. It may be used to visualize, specify, construct, and document the artifacts of a software system. It provides a set of notations (e.g. rectangles, lines, ellipses, etc.) to create a visual model of the system. Like any other language, UML has its own syntax (symbols and sentence formation rules) and semantics (meanings of symbols and sentences). Also, we should clearly understand that UML is not a system design or development methodology, but can be used to document object-oriented and analysis results obtained using some methodology.

4. Define Object Oriented Analysis?

Object Oriented Analysis (OOA) is a method of analysis that examines requirements from the perspective of the classes and objects found in the vocabulary of the problem domain.

5. What are the primary goals in the design of UML?

Provide users a ready – to use expressive visual modeling language so they can develop and exchange meaningful models.

- Provide extensibility and specialization mechanism to extend the core concepts.
- Be independent of particular programming language and development process.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of the OO tools market.
- Support higher – level development concepts.
- Integrate best practices and methodologies.

6. What is interaction diagram? Mention the types of interaction diagram.

Interaction diagrams are diagrams that describe how groups of objects collaborate to get the job done interaction diagrams capture the behavior of the single use case, showing the pattern of interaction among objects.

There are two kinds of interaction models

- Sequence Diagram
- Collaboration Diagram.

7. What is Collaboration Diagram?

Collaboration diagram represents a collaboration, which is a set of objects related in a particular context, and interaction, which is a set of messages exchanged among the objects with in collaboration to achieve a desired outcome.

8. Define Pattern?

A pattern is a named problem/solution pair that can be applied in new context, with advice on how to apply it in novel situations and discussion of its trade-offs.

9. What is Observer pattern?

The observer pattern (a subset of the publish/subscribe pattern) is a software design pattern in which an object, called the subject, maintains a list of its dependents, called observers, and notifies them automatically of any state changes, usually by calling one of their methods. It is mainly used to implement distributed event handling systems.

10. Define Start chart Diagram.

Start chart diagram shows a sequence of states that an object goes through during its life in response to events. A state is represented as a round box, which may contain one or more compartments. The compartments are all optional.

11. Define Low Coupling?

Low Coupling is an evaluative pattern, which dictates how to assign responsibilities to support:

• low dependency between classes;

• low impact in a class of changes in other classes;

• high reuse potential;

12. Define High Cohesion?

High Cohesion is an evaluative pattern that attempts to keep objects appropriately focused, manageable and understandable. High cohesion is generally used in support of Low Coupling. High cohesion means that the responsibilities of a given element are strongly related and highly focused. Breaking programs into classes and subsystems is an example of activities that increase the cohesive properties of a system.

13. What is Facade Pattern?

A facade is an object that provides a simplified interface to a larger body of code, such as a class library. A facade can:

• make a software library easier to use, understand and test, since the facade has convenient methods for common tasks;

• make code that uses the library more readable, for the same reason;

• reduce dependencies of outside code on the inner workings of a library, since most code uses the facade, thus allowing more flexibility in developing the system;

• Wrap a poorly-designed collection of APIs with a single well-designed API (as per task needs).

14. What is the main advantage of object oriented development?

- High level of abstraction
- Seamless transition among different phases of software development
- Encouragement of good programming techniques.
- Promotion of reusability.

15. What do you mean by information hiding?

Information hiding is the principle of concealing the internal data and procedures of an object and providing an interface to each object and providing an interface to each object in such a way as to reveal as little as possible about its inner workings.

- An object is often said to encapsulate the date and a program.
- Per-class protection.

• Class methods can access any object of that class and not just the receiver

- Per-object protection.

• Methods can access only the receiver.

16. Explain object relationship and associations?

a. Association represents the relationships between objects and classes.

b. Associations are bidirectional.

c. Cardinality specifies how many instances of one class may relate to a single instance of an associated class.

d. Cardinality constraints the number of related objects and often is described as being "one" or "many"

PART-B(11 MARKS)

OBJECT MODELLING USING UML

- The object-oriented software development Style has become extremely popular, and is being widely used in industry as well as in academic circles.
- In the context of model construction we need to carefully understand the basic distinction between a modeling language and a design process.
- A modeling language specifies a set of notations using which a design or analysis result can be documented.
- A design process address the starting from a given problem description that is how exactly can one work out the design solution to the problem? In other words, a design process recommends a step by step procedure using a problem description can be converted to a design solution a design process is referred as design methodology.

OVERVIEW OF BASIC OBJECT-ORIENTATION CONCEPTS

The principles of object-orientation are founded on few important concepts. The important concepts and mechanisms are based on which the principles of object-orientation have been founded.

Basic Mechanisms
The following are the important mechanisms used in the object-oriented paradigm.

Objects:

It is convenient to think of each object has representing a real world entity such as library member, employee or a book etc.,
A key advantage of considering a system as a set of objects is the following,
When the system is analyzed, developed, and implemented in terms of natural objects, it becomes easy to understand the design and the implementation of the system.
Each object stores some data and supports certain operations on the stored data.
For example, Consider library member object can be:

- Name of the Member
- Membership number
- Address
- Phone number
- Email

The operations supported by a library member object can be:

- Issue-book
- Find-books-outstanding
- Find-books-overdue
- Return-book
- Find-membership-details

The data stored internally in an object are called its attributes and the functions supported by an object are called in methods.

Class:

Similar objects constitute a class. This means that objects possessing similar attributes and having similar methods would constitute a class. For example, the set of all the library members would constitute a class in a library automation application. In this case, each library member object has attributes such as member name, membership number,member address,etc. And also methods such as issue-book, return-book, etc.

A class can be considered as an abstract data type (ADT). A class being an ADT has the following implications:

- The data of an object can be accessed only through its methods. In other words, the way data is stored internally (stack, array, queue, etc.) in the object is abstracted out(not known to other objects).
- We can instantiate a class into objects(i.e. a class is a type).

Methods

1. The operations(such as create, issue, return, etc.) supported by an object are implemented in the form of methods. The terms operation and 'method' are often used interchangeably. However, there is a technical difference between these two terms which we explain in the following section.
2. A method is an implementation of the responsibility. It would be sometimes useful to implement a responsibility through more than one method.
3. When an operation is implemented through multiple methods, we say that the method name is overloaded.
4. The implementation of a class responsibility through multiple methods is known as method overloading.

Class relationships

Classes can be related to each other in the following four ways

- Inheritance
- Association
- Aggregation and composition
- Dependency

In the following, we discuss these different types of relationships that may exist among classes.

Inheritance

1. The inheritance feature allows us to define a new class by extending the features of modifying an existing class.

2. The original class is called the base class and the new class obtained through inheritance is called derived class.
3. An example of inheritance is the classes Faculty, Students, and staff as having been derived from the base class LibraryMember through an inheritance relationship.
4. Each derived class can be considered as a specialization of its base class because it modifies or extends thebasic properties of the base class in certain wa
5. A derived class may even give a new definition to a method which already exists in the base class. Redefinition of a method which already exists in its base class is termed as method overriding.
6. When a new definition of a method that existed in the base class is provided in a derived class, the method is said to be overridden in the derived class.
7. Inheritance is a basic mechanism that almost every object-oriented programming language supports in fact, languages that support ADTs, but do not support inheritance are called object based languages.
8. The important advantage of using the inheritance mechanism in programming include code reuse and simplicity of program design.

Multiple Inheritance

Multiple inheritance is a mechanism by which a subclass can inherit attributes and methods from more than one base class.

Association and Link

1. If one class is associated with another, then the corresponding objects of two classes know each other.
2. Each object of one class in the association relationships knows the address of the corresponding object of the other class.As a result, it becomes possible for the object of one class to invoke the methods of the corresponding object of the other class.
3. When two classes are associated the relationship between two objects of the two corresponding class is called a link.
4. In other words, an association describes a group of similar links. A link can be considered has an instance of an association relation.
5. In case of recursive association, two or more different objects of the same class are linked by association relationship.

Composition and Aggregation

1. Composition and Aggregation represent whole or part relationships among different objects. Objects which contains other objects are called composite objects.

For example, Suppose a student can register in five courses in a semester in this case a studentSemesterRegistration object is composed of five course object.For this reason, the composition/aggregation relationship is also known as has relationship. Composition can occur in a hierarchy of levels.

2. The composition and aggregation relationship are not reflexive. In other words, association relationship cannot be circularly defined. An object cannot contain the object of same type.

Dependency

A dependency relation between two classes shows that any change made to the independent class would require the corresponding change to be made to the independent class. Independencies among classes may arise due to various causes. There are two important reasons as follows:

- A class invokes the method provided by another class
- A class implements an interface class. If the properties of the interface class are changed, then a change becomes necessary to the class implementing the interface class as well.

Abstract Class

1. Classes that are not intended to produce instances of themselves are called abstract classes. In other words, abstract classes cannot be instantiated.
2. By using abstract classes, code reuse can be enhanced and the effort required to develop software brought down.
3. Abstract classes usually supports generic methods, and the subclasses of the abstract classes are expected to provide specific implementation of these methods.

1.1.2 KEY CONCEPTS

Abstraction

Abstraction is the selective examination of certain aspects of a problem while ignoring all the remaining aspects of a problem. The abstraction mechanism allows us to represent a problem in a simpler way by considering only those aspects that are relevant to some purpose and omitting all other details that are irrelevant.

Feature Abstraction

A class hierarchy can be viewed as defining several levels of abstraction, where each class is an abstraction of its subclasses.

Data Abstraction

An object itself can be considered as a data abstraction entity, because it abstracts out the exact way in which it stores its various private data items, and it merely provides a set of methods to other objects to access and manipulate these data items.An important advantage of the principle of data abstraction is that it reduces coupling among various objects, Therefore, it leads to a reduction of the overall complexity of a design, and helps in easy maintenance and code reuse.

Encapsulation

The data of an object is encapsulated within its methods. To access the data internal to an object, other objects have to invoke its methods, and cannot directly access the data.
Encapsulation offers three important advantages as follows:

- Protection from unauthorized data access
- Data hiding
- Weak coupling

Polymorphism

Polymorphism literally means poly(many) morphism(forms). Polymorphism can be implemented using the following two mechanisms:

1. Static polymorphism:

1. In this type of polymorphism, the same method call results in different actions. This type of polymorphism is also referred to as static binding.

2. Static polymorphism occurs when multiple methods with same operation name exist.

2. Dynamic polymorphism:

In dynamic binding, address of an invoked method can be known only at run time. It is important to remember that dynamic binding occurs when we have some methods of the base class overridden by the derived classes and the instances of the derived classes are stored in instances of the base class.

Even when the method of an object of the base class is invoked, an appropriate overridden method of a derived class would be invoked depending on the exact object that may have been assigned at the run-time to the object of the base class.

The main advantage of dynamic binding is that it leads to elegant programming and facilitates code reuse and maintenance.

1.1.3 RELATED TECHNICAL TERMS

Persistence

Objects usually get destroyed once a program finishes execution. Persistent objects are stored permanently.

Agents

A passive object is one that performs some action only when requested through invocation of some its methods. An agent on the other hand, monitors events occurring in the application and takes actions autonomously.

Widgets

The term widget stands for window object. A widget is a primitive object used for graphical user interface(GUI) design.

1.1.4 ADVANTAGES OF OOD

The main reason for the popularity of OOD is that it holds the following promises:

- Code and design reuse
- Increased productivity
- Ease of testing and maintenance
- Better code and design understandability

The chief advantage of OOD is improved productivity which comes about due to a variety of factors, such as

- Code reuse by the use of pre developed class libraries
- Code reuse due to inheritance
- Simpler and more intuitive abstraction, i.e. better organization of inherent complexity
- Better problem decomposition

UNIFIED MODELING LANGUAGE (UML)

UML, as the name implies, is a modeling language. It may be used to visualize, specify, construct, and document the artifacts of a software system. It provides a set of notations (e.g. rectangles, lines, ellipses, etc.) to create a visual model of the system. Like any other language, UML has its own syntax (symbols and sentence formation rules) and semantics (meanings of symbols and sentences). Also, we should clearly understand that UML is not a system design or development methodology, but can be used to document object-oriented and analysis results obtained using some methodology.

Origin of UML

In the late 1980s and early 1990s, there was a proliferation of object-oriented design techniques and notations. Different software development houses were using different notations to document their object-oriented designs. These diverse notations used to give rise to a lot of confusion.

UML was developed to standardize the large number of object-oriented modeling notations that existed and were used extensively in the early 1990s. The principles ones in use were:

- Object Management Technology [Rumbaugh 1991]
- Booch's methodology [Booch 1991]
- Object-Oriented Software Engineering [Jacobson 1992]
- Odell's methodology [Odell 1992]
- Shaler and Mellor methodology [Shaler 1992]

It is needless to say that UML has borrowed many concepts from these modeling techniques. Especially, concepts from the first three methodologies have been heavily drawn upon. UML was adopted by Object Management Group (OMG) as a *de facto* standard in 1997. OMG is an association of industries which tries to facilitate early formation of standards.

We shall see that UML contains an extensive set of notations and suggests construction of many types of diagrams. It has successfully been used to model both large and small problems. The elegance of UML, its adoption by OMG, and a strong industry backing have helped UML find widespread acceptance. UML is now being used in a large number of software development projects worldwide.

What is a model?

A model is an abstraction of a real problem (or situation), and is constructed by leaving out unnecessary details. This reduces the problem complexity and makes it easy to understand the problem(or situation).

Why construct a model?

An important reason behind constructing a model is that it helps to manage the complexity in a problem and facilities arriving at good solutions and at the same time helps to reduce the design cost.

Once models of a system have been constructed, these can be used for a variety of purposes during software development. Including the following:

- Analysis
- Specification
- Design
- Coding
- Visualization and understanding of an implementation
- Testing, etc.,

3. UML DIAGRAMS

UML can be used to construct nine different types of diagrams to capture five different views of a system. Just as a building can be modeled from several views (or perspectives) such as ventilation perspective, electrical perspective, lighting perspective, heating perspective, etc.; the different UML diagrams provide different perspectives of the software system to be developed and facilitate a comprehensive understanding of the system. Such models can be refined to get the actual implementation of the system.

The UML diagrams can capture the following five views of a system**:**

• User's view
• Structural view
• Behavioral view
• Implementation view
• Environmental view

Fig. shows the UML diagrams responsible for providing the different views.

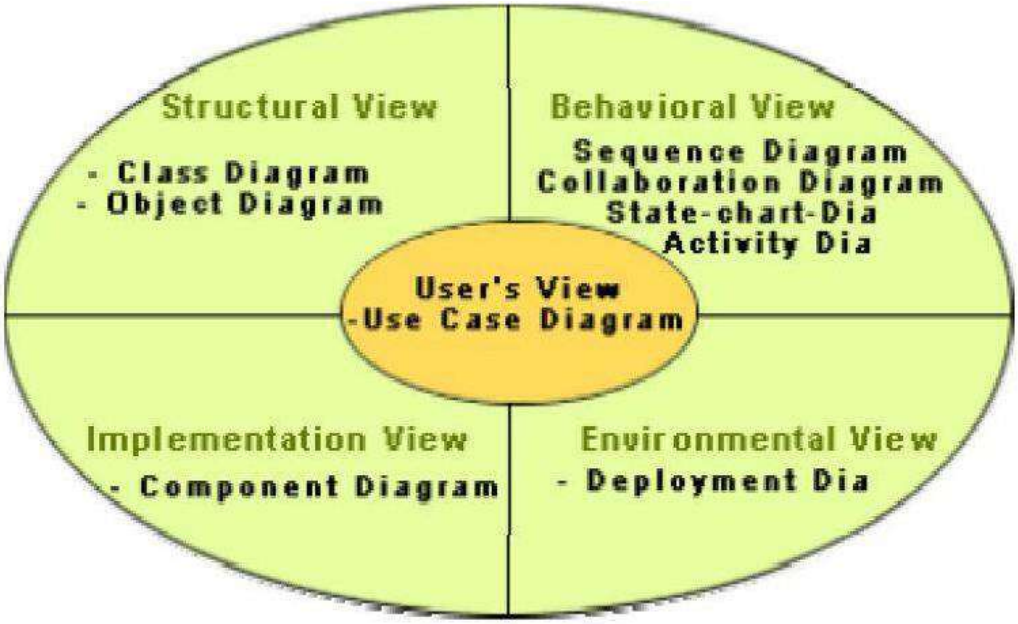


Fig. Different types of diagrams and views supported in UML

1. **User's view:** This view defines the functionalities (facilities) made available by the system to its users. The users' view captures the external users' view of the system in terms of the functionalities offered by the system. The users' view is a black-box view of the system where the internal structure, the dynamic behavior of different system components, the implementation etc. are not visible. The users' view is very different from all other views in the sense that it is a functional model compared to the object model of all other views. The users' view can be considered as the central view and all other views are expected to conform to this view. This thinking is in fact the crux of any user centric development style.
2. **Structural view:** The structural view defines the kinds of objects (classes) important to the understanding of the working of a system and to its implementation. It also captures the relationships among the classes (objects). The structural model is also called the static model, since the structure of a system does not change with time.
3. **Behavioral view:** The behavioral view captures how objects interact with each other to realize the system behavior. The system behavior captures the time-dependent (dynamic) behavior of the system.
4. **Implementation view:** This view captures the important components of the system and their dependencies.
5. **Environmental view:** This view models how the different components are implemented on different pieces of hardware.

USE CASE MODEL

The use case model for any system consists of a set of "use cases". Intuitively, use cases represent the different ways in which a system can be used by the users. A simple way to find all the use cases of a system is to ask the question: "What the users can do using the system?" Thus for the Library Information System (LIS), the use cases could be:

- issue-book
- query-book
- return-book
- create-member
- add-book, etc

Use cases correspond to the high-level functional requirements. The use cases partition the system behavior into transactions, such that each transaction performs some useful action from the user's point of view. To complete each transaction may involve either a single message or multiple message exchanges between the user and the system to complete.

Purpose of use cases

The purpose of a use case is to define a piece of coherent behavior without revealing the internal structure of the system. The use cases do not mention any specific algorithm to be used or the internal data representation, internal structure of the software, etc. A use case typically represents a sequence of interactions between the user and the system. These interactions consist of one mainline sequence.

The mainline sequence represents the normal interaction between a user and the system. The mainline sequence is the most occurring sequence of interaction. For example, the mainline sequence of the withdraw cash use case supported by a bank ATM drawn, complete the transaction, and get the amount. Several variations to the main line sequence may also exist.

Typically, a variation from the mainline sequence occurs when some specific conditions hold. For the bank ATM example, variations or alternate scenarios may occur, if the password is invalid or the amount to be withdrawn exceeds the amount balance. The variations are also called alternative paths.

A use case can be viewed as a set of related scenarios tied together by a common goal. The mainline sequence and each of the variations are called scenarios or instances of the use case. Each scenario is a single path of user events and system activity through the use case.

Representation Of Use Cases

Use cases can be represented by drawing a use case diagram and writing an accompanying text elaborating the drawing. In the use case diagram, each use case is represented by an ellipse with the name of the use case written inside the ellipse. All the ellipses (i.e. use cases) of a system are enclosed within a rectangle which represents the system boundary. The name of the system being modeled (such as Library Information System) appears inside the rectangle.

The different users of the system are represented by using the stick person icon. Each stick person icon is normally referred to as an actor. An actor is a role played by a user with respect to the system use. It is possible that the same user may play the role of multiple

actors. Each actor can participate in one or more use cases. The line connecting the actor and the use case is called the communication relationship. It indicates that the actor makes use of the functionality provided by the use case. Both the human users and the external systems can be represented by stick person icons. When a stick person icon represents an external system, it is annotated by the stereotype <<external system>>.

Example 1:

The use case model for the Tic-tac-toe problem is shown in fig. This software has only one use case "play move". Note that the use case "get-user-move" is not used here. The name "get-user-move" would be inappropriate because the use cases should be named from the users' perspective.

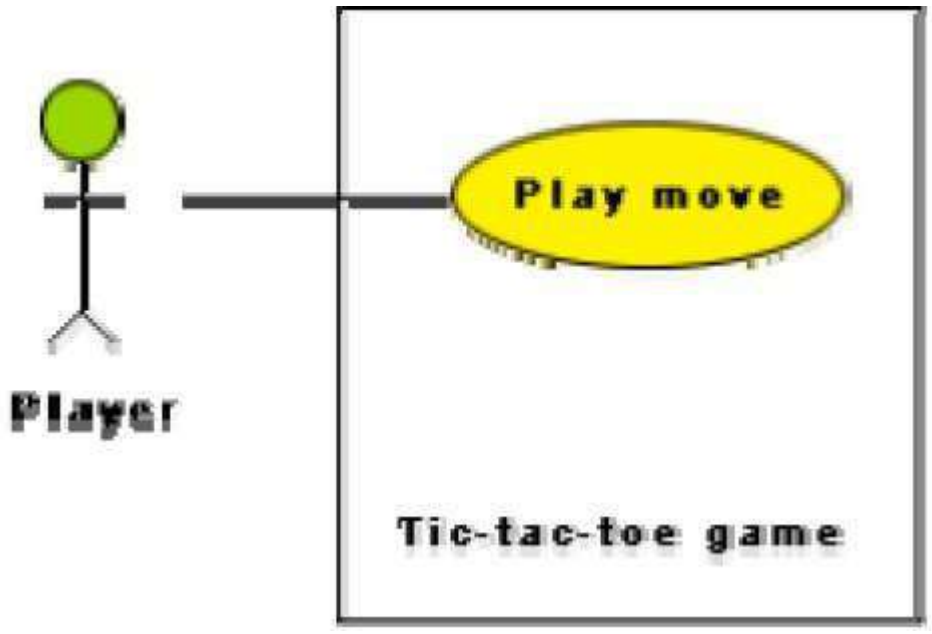


Fig.: Use case model for tic-tac-toe game **Text Description**

Each ellipse on the use case diagram should be accompanied by a text description. The text description should define the details of the interaction between the user and the computer and other aspects of the use case. It should include all the behavior associated with the use case in terms of the mainline sequence, different variations to the normal behavior, the system responses associated with the use case, the exceptional conditions that may occur in the behavior, etc. The behavior description is often written in a conversational style describing the interactions between the actor and the system. The text description may be informal, but some structuring is recommended. The following are some of the information which may be included in a use case text description in addition to the mainline sequence, and the alternative scenarios.

1. **Contact persons:** This section lists the personnel of the client organization with whom the use case was discussed, date and time of the meeting, etc.

2. **Actors:** In addition to identifying the actors, some information about actors using this use case which may help the implementation of the use case may be recorded.
3. **Pre-condition:** The preconditions would describe the state of the system before the use case execution starts.
4. **Post-condition:** This captures the state of the system after the use case has successfully completed.
5. **Non-functional requirements:** This could contain the important constraints for the design and implementation, such as platform and environment conditions, qualitative statements, response time requirements, etc.
6. **Exceptions, error situations:** This contains only the domain-related errors such as lack of user's access rights, invalid entry in the input fields, etc. Obviously, errors that are not domain related, such as software errors, need not be discussed here.
7. **Sample dialogs:** These serve as examples illustrating theuse case.
8. **Specific user interface requirements:** These contain specific requirements for the user interface of the use case. For example, it may contain forms to be used, screen shots, interaction style, etc.
9. **Document references:** This part contains references to specific domain-related documents which may be useful to understand the system operation.

Example 2:

The use case model for the Supermarket Prize Scheme described in Lesson 5.2 is shown in fig. As discussed earlier, the use cases correspond to the high-level functional requirements. From the problem description and the context diagram in fig., we can identify three use cases: "register-customer", "register-sales", and "select-winners". As a sample, the text description for the use case "register-customer" is shown.

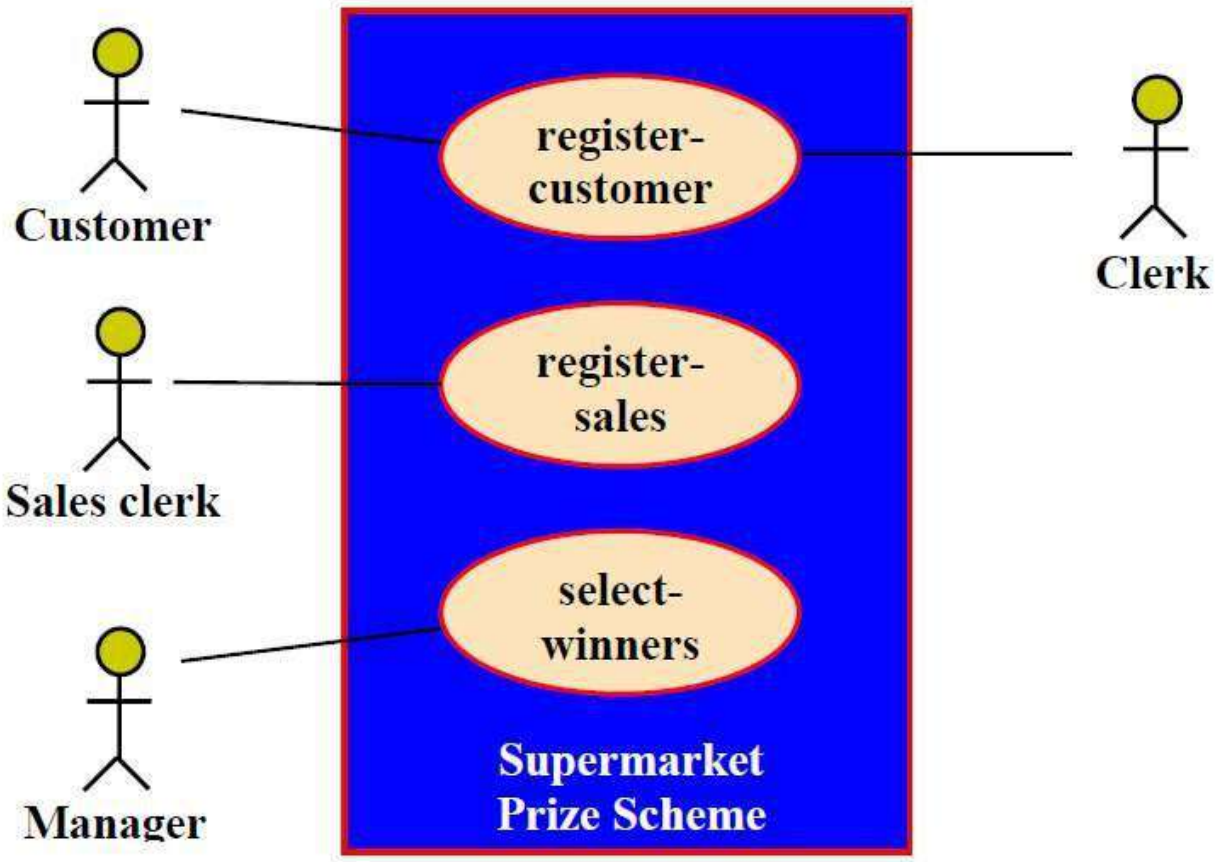


Fig. Use case model for Supermarket Prize Scheme **register-customerregister-salesselect-winnersSupermarket PrizeSchemeCustomer Sales clerk Manager Clerk**

Text description

U1: register-customer: Using this use case, the customer can register himself by providing the necessary details.

Scenario 1: Mainline sequence

1. **Customer: select register customer option.**
2. **System: display prompt to enter name, address, and telephone number.**
3. **Customer: enter the necessary values.**
4. **System: display the generated id and the message that the customer has been successfully registered.**

Scenario 2: at step 4 of mainline sequence

1. **System: displays the message that the customer has already registered.**

Scenario 2: at step 4 of mainline sequence

1. **System: displays the message that some input information has not been entered. The system display a prompt to enter the missing value.**

The description for other use cases is written in a similar fashion.

Utility of use case diagrams

From use case diagram, it is obvious that the utility of the use cases are represented by ellipses. They along with the accompanying text description serve as a type of requirements specification of the system and form the core model to which all other models must conform. But, what about the actors (stick person icons)? One possible use of identifying the different types of users (actors) is in identifying and implementing a security mechanism through a login system, so that each actor can involve only those functionalities to which he is entitled to. Another possible use is in preparing the documentation (e.g. users' manual) targeted at each category of user. Further, actors help in identifying the use cases and understanding the exact functioning of the system.

Factoring of use cases

It is often desirable to factor use cases into component use cases. Actually, factoring of use cases are required under two situations. First, complex use cases need to be factored into simpler use cases. This would not only make the behavior associated with the use case much more comprehensible, but also make the corresponding interaction diagrams more tractable. Without decomposition, the interaction diagrams for complex use cases may become too large to be accommodated on a single sized (A4) paper.

Secondly, use cases need to be factored whenever there is common behavior across different use cases. Factoring would make it possible to define such behavior only once and reuse it whenever required. It is desirable to factor out common usage such as error handling from a set of use cases. This makes analysis of the class design much simpler and elegant. However, a word of caution here. Factoring of use cases should not be done except for achieving the above two objectives. From the design point of view, it is not advantageous to break up a use case into many smaller parts just for the shake of it.

UML offers three mechanisms for factoring of use cases as follows:

1. Generalization

Use case generalization can be used when one use case that is similar to another, but does something slightly differently or something more. Generalization works the same way with use cases as it does with classes. The child use case inherits the behavior and meaning of the parent use case. The notation is the same too (as shown in fig.). It is important to remember that the base and the derived use cases are separate use cases and should have separate text descriptions.

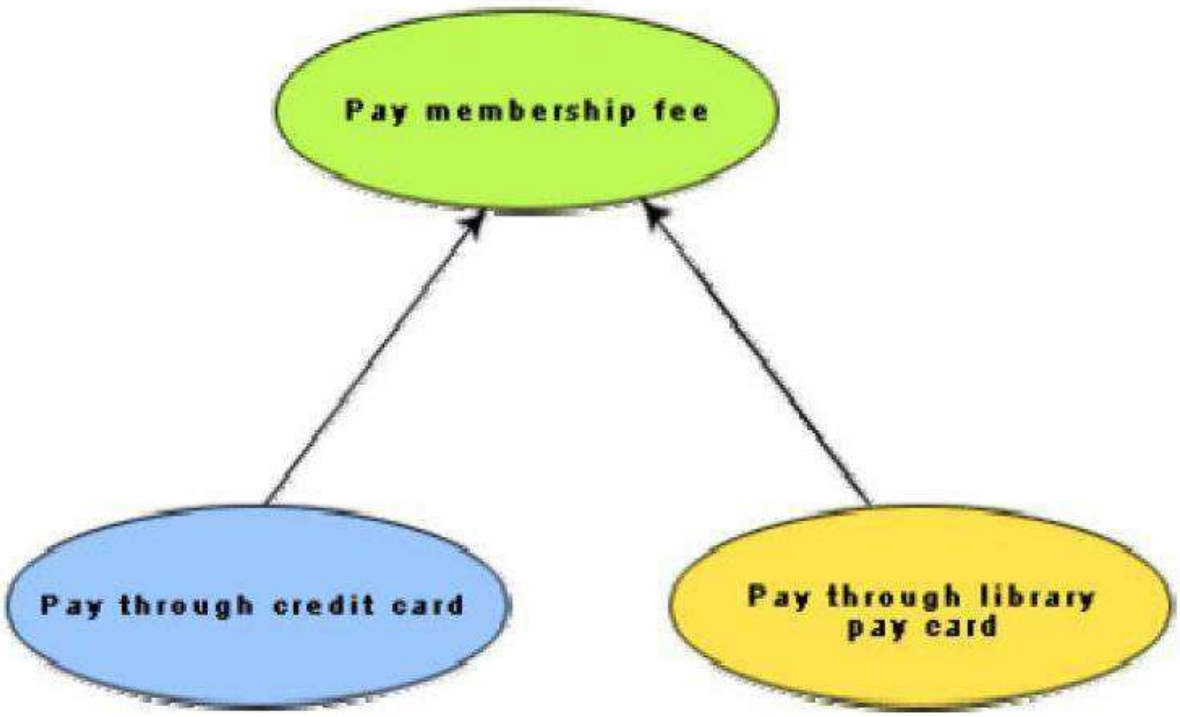


Fig.: Representation of use case generalization

2. Includes

The includes relationship in the older versions of UML (prior to UML 1.1) was known as the uses relationship. The includes relationship involves one use case including the behavior of another use case in its sequence of events and actions.

The includes relationship occurs when a chunk of behavior that is similar across a number of use cases. The factoring of such behavior will help in not repeating the specification and implementation across different use cases. Thus, the includes relationship explores the issue of reuse by factoring out the commonality across use cases. It can also be gainfully employed to decompose a large and complex use cases into more manageable parts. As shown in fig, the includes relationship is represented using a predefined stereotype <<include>>. In the includes relationship, a base use case compulsorily and automatically includes the behavior of the common use cases. As shown in example fig., issue-book and renew-book both include check-reservation use case. The base use case may include several use cases. In such cases, it may interleave their associated common use cases together. The common use case becomes a separate use case and the independent text description should be provided for it.

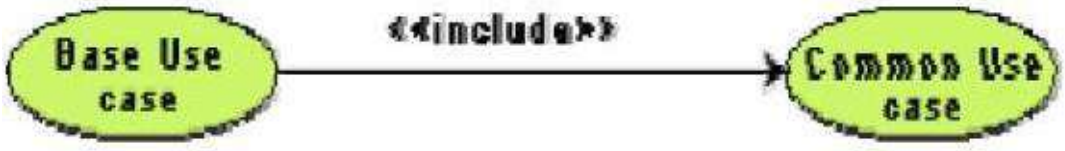


Fig. 7.5: Representation of use case inclusion

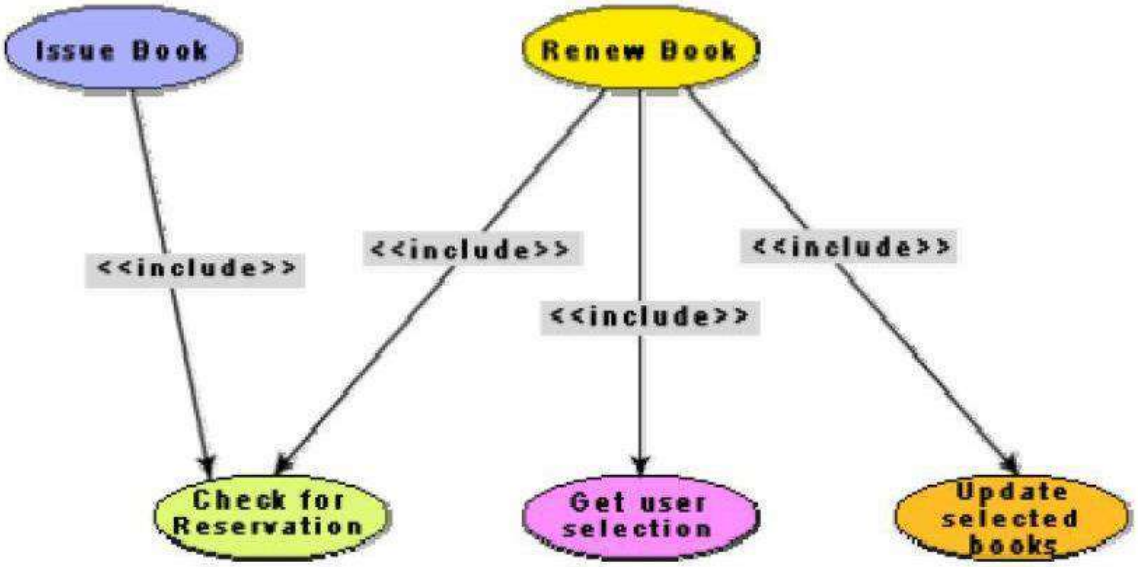


Fig. 7.6: Example use case inclusion

3. Extends

The main idea behind the extends relationship among the use cases is that it allows you to show optional system behavior. An optional system behavior is extended only under certain conditions. This relationship among use cases is also predefined as a stereotype as shown in fig. 7.7. The extends relationship is similar to generalization. But unlike generalization, the extending use case can add additional behavior only at an extension point only when certain conditions are satisfied. The extension points are points within the use case where variation to the mainline (normal) action sequence may occur. The extends relationship is normally used to capture alternate paths or scenarios.

Fig. 7.7: Example use case extension

4. Organization of use cases

When the use cases are factored, they are organized hierarchically. The high-level use cases are refined into a set of smaller and more refined use cases as shown in fig. 7.8. Top-level use cases are super-ordinate to the refined use cases. The refined use cases are sub-ordinate to the top-level use cases. Note that only the complex use cases should be decomposed and organized in a hierarchy. It is not necessary to decompose simple use cases.

The functionality of the super-ordinate use cases is traceable to their sub-ordinate use cases. Thus, the functionality provided by the super-ordinate use cases is composite of the functionality of the sub-ordinate use cases. In the highest level of the use case model, only the fundamental use cases are shown. The focus is on the application context.

Therefore, this level is also referred to as the context diagram. In the context diagram, the system limits are emphasized. In the top-level diagram, only those use cases with which external users of the system. The subsystem-level use cases specify the services offered by the subsystems. Any number of levels involving the subsystems may be utilized. In the lowest level of the use case hierarchy, the class-level use cases specify the functional fragments or operations offered by the classes.

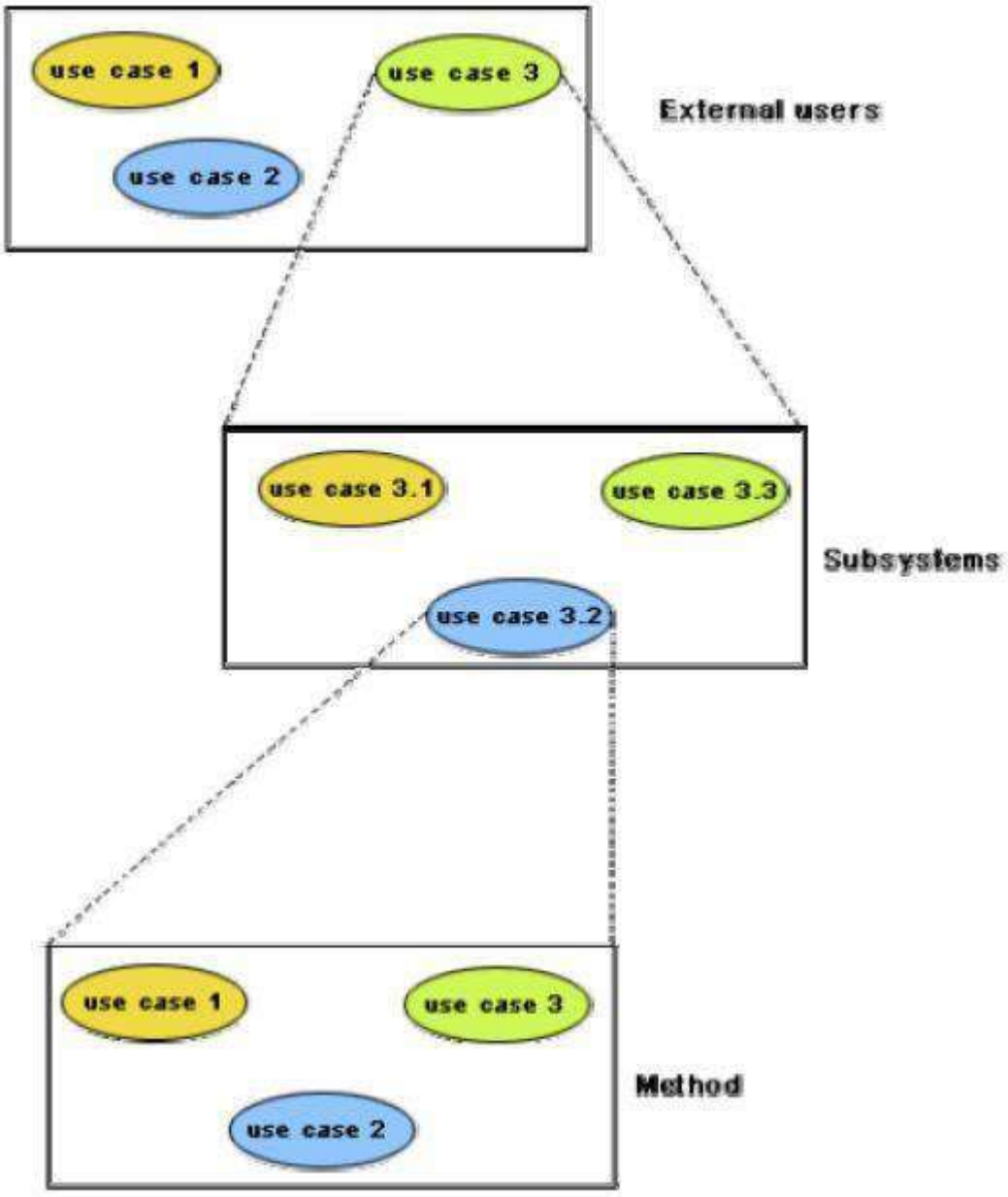


Fig.: Hierarchical organization of use cases

CLASS DIAGRAMS

A class diagram describes the static structure of a system. It shows how a system is structured rather than how it behaves. The static structure of a system comprises of a number of class diagrams and their dependencies. The main constituents of a class diagram are classes and their relationships: generalization, aggregation, association, and various kinds of dependencies.

Classes

The classes represent entities with common features, i.e. attributes and operations. Classes are represented as solid outline rectangles with compartments. Classes have a mandatory name compartment where the name is written centered in boldface. The class name is usually written using mixed case convention and begins with an uppercase. The class names are usually chosen to be singular nouns. An example of a class is shown in fig. 6.2 (Lesson 6.1).

Classes have optional attributes and operations compartments. A class may appear on several diagrams. Its attributes and operations are suppressed on all but one diagram.

Attributes

An attribute is a named property of a class. It represents the kind of data that an object might contain. Attributes are listed with their names, and may optionally contain specification of their type, an initial value, and constraints. The type of the attribute is written by appending a colon and the type name after the attribute name. Typically, the first letter of a class name is a small letter. An example for an attribute is given.

bookName : String

Operation

Operation is the implementation of a service that can be requested from any object of the class to affect behaviour. An object's data or state can be changed by invoking an operation of the object. A class may have any number of operations or no operation at all. Typically, the first letter of an operation name is a small letter. Abstract operations are written in italics. The parameters of an operation (if any), may have a kind specified, which may be 'in', 'out' or 'inout'. An operation may have a return type consisting of a single return type expression. An example for an operation is given.

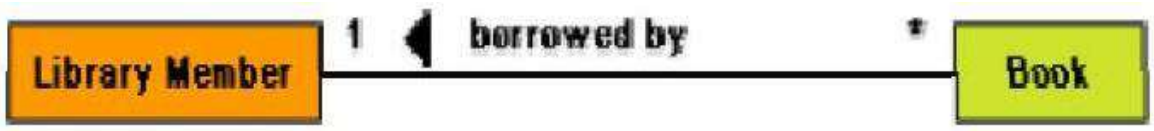
issueBook(in bookName):Boolean

Association

Associations are needed to enable objects to communicate with each other. An association describes a connection between classes. The association relation between two objects is called object connection or link. Links are instances of associations. A link is a physical or conceptual connection between object instances. For example, suppose Amit has borrowed the book Graph Theory. Here, borrowed is the connection between the objects Amit and Graph Theory book. Mathematically, a link can be considered to be a tuple, i.e. an ordered list of object instances. An association describes a group of links with a common structure and common semantics. For example, consider the statement that Library Member borrows Books. Here,

borrows is the association between the class LibraryMember and the class Book. Usually, an association is a binary relation (between two classes). However, three or more different classes can be involved in an association. A class can have an association relationship with itself (called recursive association). In this case, it is usually assumed that two different objects of the class are linked by the association relationship.

Association between two classes is represented by drawing a straight line between the concerned classes. Fig. 7.9 illustrates the graphical representation of the association relation. The name of the association is written along side the association line. An arrowhead may be placed on the association line to indicate the reading direction of the association. The arrowhead should not be misunderstood to be indicating the direction of a pointer implementing an association. On each side of the association relation, the multiplicity is noted as an individual number or as a value range. The multiplicity indicates how many instances of one class are associated with each other. Value ranges of multiplicity are noted by specifying the minimum and maximum value, separated by two dots, e.g. 1.5. An asterisk is a wild card and means many (zero or more). The association of fig. 7.9 should be read as "Many books may be borrowed by a Library Member". Observe that associations (and links) appear as verbs in the problem statement.



Association between two classes

Associations are usually realized by assigning appropriate reference attributes to the classes involved. Thus, associations can be implemented using pointers from one object class to another. Links and associations can also be implemented by using a separate class that stores which objects of a class are linked to which objects of another class. Some CASE tools use the role names of the association relation for the corresponding automatically generated attribute.

Aggregation

Aggregation is a special type of association where the involved classes represent a whole-part relationship. The aggregate takes the responsibility of forwarding messages to the appropriate parts. Thus, the aggregate takes the responsibility of delegation and leadership. When an instance of one object contains instances of some other objects, then aggregation (or composition) relationship exists between the composite object and the component object. Aggregation is represented by the diamond symbol at the composite end of a relationship. The number of instances of the component class aggregated can also be shown as in fig.

Fig.: Representation of aggregation

Aggregation relationship cannot be reflexive (i.e. recursive). That is, an object cannot contain objects of the same class as itself. Also, the aggregation relation is not symmetric. That is, two classes A and B cannot contain instances of each other. However, the aggregation relationship can be transitive. In this case, aggregation may consist of an arbitrary number of levels.

Composition

Composition is a stricter form of aggregation, in which the parts are existence-dependent on the whole. This means that the life of the parts closely ties to the life of the whole. When the whole is created, the parts are created and when the whole is destroyed, the parts are destroyed. A typical example of composition is an invoice object with invoice items. As soon as the invoice object is created, all the invoice items in it are created and as soon as the invoice object is destroyed, all invoice items in it are also destroyed. The composition relationship is represented as a filled diamond drawn at the composite-end. An example of the composition relationship is shown in fig.

Fig.: Representation of composition

Association vs. Aggregation vs. Composition

- Association is the most general (m:n) relationship. Aggregation is a stronger relationship where one is a part of the other. Composition is even stronger than aggregation, ties the lifecycle of the part and the whole together.
- Association relationship can be reflexive (objects can have relation to itself), but aggregation cannot be reflexive. Moreover, aggregation is anti-symmetric (If B is a part of A, A can not be a part of B).
- Composition has the property of exclusive aggregation i.e. an object can be a part of only one composite at a time. For example, a **Frame** belongs to exactly one **Window** whereas in simple aggregation, a part may be shared by several objects. For example, a **Wall** may be a part of one or more **Room** objects.
- In addition, in composition, the whole has the responsibility for the disposition of all its parts, i.e. for their creation and destruction.

    - o in general, the lifetime of parts and composite coincides
    - o parts with non-fixed multiplicity may be created after composite itself
    - o parts might be explicitly removed before the death of the composite

For example, when a **Frame** is created, it has to be attached to an enclosing **Window**. Similarly, when the **Window** is destroyed, it must in turn destroy its **Frame** parts.

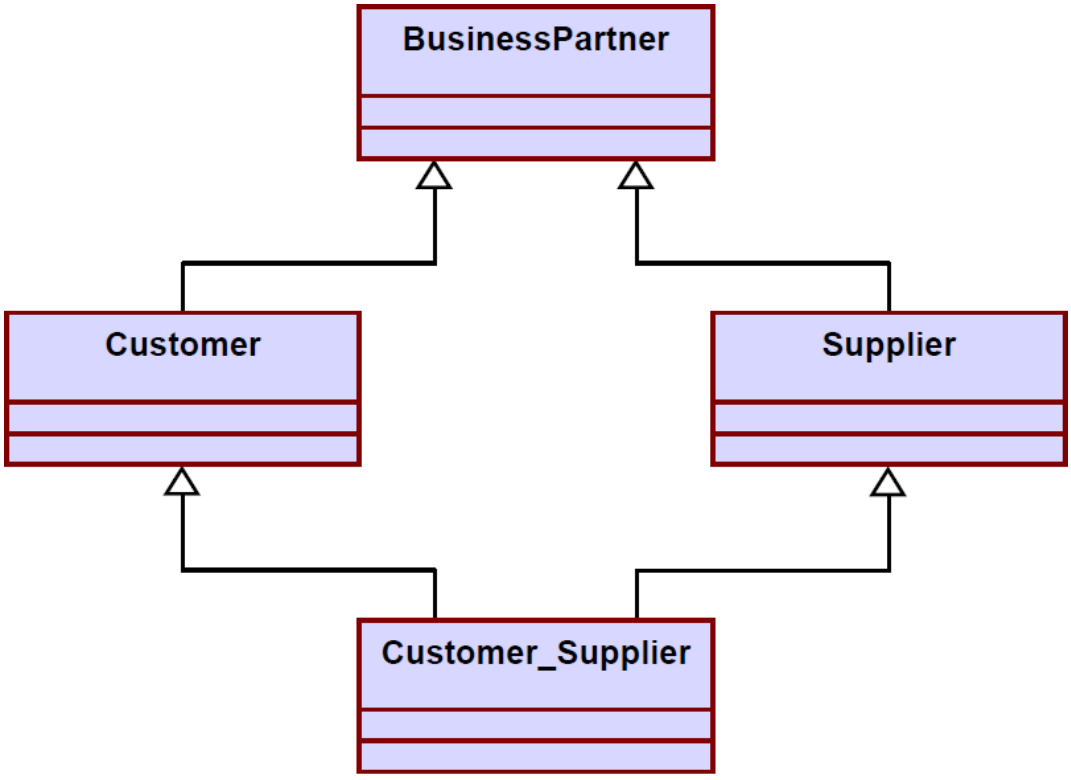
Inheritance vs. Aggregation/Composition

- Inheritance describes *'is a' / 'is a kind of'* relationship between classes (base class - derived class) whereas aggregation describes *'has a'* relationship between classes. Inheritance means that the object of the derived class inherits the properties of the base class; aggregation means that the object of the whole has objects of the part. For example, the relation "cash payment *is a kind of* payment" is modeled using inheritance; "purchase order has a few items" is modeled using aggregation.
  Inheritance is used to model a "generic-specific" relationship between classes whereas aggregation/composition is used to model a "whole-part" relationship between classes.

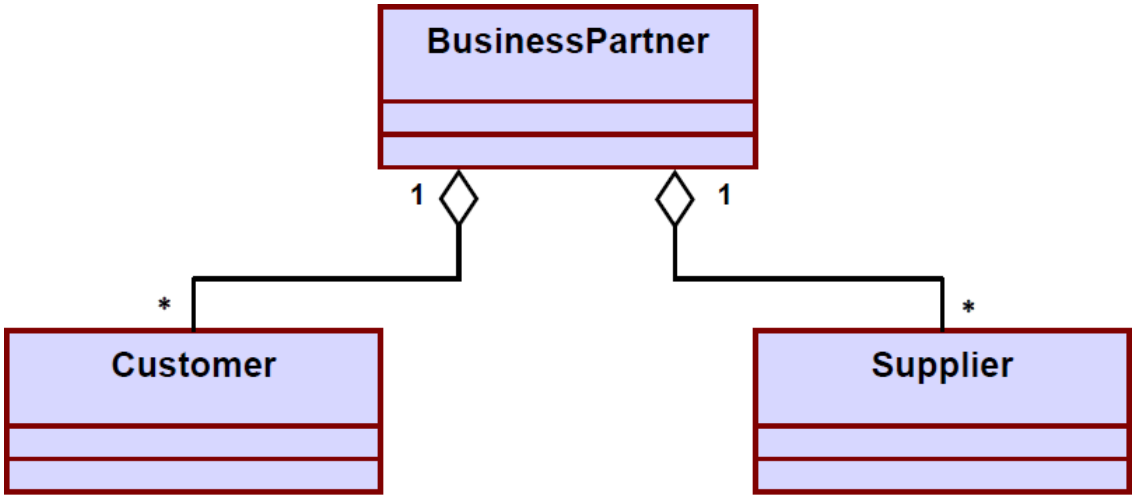
- Inheritance means that the objects of the subclass can be used anywhere the super class may appear, but not the reverse; i.e. wherever we could use instances of 'payment' in the system, we could substitute it with instances of 'cash payment', but the reverse can not be done.

- Inheritance is defined statically. It can not be changed at run-time. Aggregation is defined dynamically and can be changed at run-time. Aggregation is used when the type of the object can change over time.
  For example, consider this situation in a business system. A **BusinessPartner** might be a **Customer** or a **Supplier** or both. Initially we might be tempted to model it as in Fig 7.12(a). But in fact, during its lifetime, a business partner might become a customer as well as a supplier, or it might change from one to the other. In such cases, we prefer aggregation instead (see Fig 7.12(b). Here, a business partner is a **Customer** if it has an aggregated **Customer** object, a **Supplier** if it has an aggregated **Supplier** object and a "C**ustomer_Supplier**" if it has both. Here, we have only two types. Hence, we are able to model it as inheritance. But what if there were several different types and combinations there of? The inheritance tree would be absolutely incomprehensible.
  Also, the aggregation model allows the possibility for a business partner to be neither - i.e. has neither a customer nor a supplier object aggregated with it.

- The advantage of aggregation is the integrity of encapsulation. The operations of an object are the interfaces of other objects which imply low implementation dependencies. The significant disadvantage of aggregation is the increase in the number of objects and their relationships. On the other hand, inheritance allows for an easy way to modify implementation for reusability. But the significant disadvantage is that it breaks encapsulation, which implies implementation dependence.



(a)



(b)

Fig. Representation of **BusinessPartner, Customer, Supplier** relationship **(a)** using inheritance **(b)** using aggregation

INTERACTION DIAGRAMS

Interaction diagrams are models that describe how group of objects collaborate to realize some behavior. Typically, each interaction diagram realizes the behavior of a single use case. An interaction diagram shows a number of example objects and the messages that are passed between the objects within the use case.

There are two kinds of interaction diagrams: sequence diagrams and collaboration diagrams. These two diagrams are equivalent in the sense that any one diagram can be derived automatically from the other. However, they are both useful. These two actually portray different perspectives of behavior of the system and different types of inferences can be drawn from them. The interaction diagrams can be considered as a major tool in the design methodology.

Sequence Diagram

A sequence diagram shows interaction among objects as a two dimensional chart. The chart is read from top to bottom. The objects participating in the interaction are shown at the top of the chart as boxes attached to a vertical dashed line. Inside the box the name of the object is written with a colon separating it from the name of the class and both the name of the object and the class are underlined. The objects appearing at the top signify that the object already existed when the use case execution was initiated. However, if some object is created during the execution of the use case and participates in the interaction (e.g. a method call), then the object should be shown at the appropriate place on the diagram where it is created. The vertical dashed line is called the object's lifeline. The lifeline indicates the existence of the object at any particular point of time. The rectangle drawn on the lifetime is called the activation symbol and indicates that the object is active as long as the rectangle exists. Each message is indicated as an arrow between the lifeline of two objects. The messages are shown in chronological order from the top to the bottom. That is, reading the diagram from the top to the bottom would show the sequence in which the messages occur. Each message is labeled with the message name. Some control information can also be included. Two types of control information are particularly valuable.

- A condition (e.g. [invalid]) indicates that a message is sent, only if the condition is true.
- An iteration marker shows the message is sent many times to multiple receiver objects as would happen when a collection or the elements of an array are being iterated. The basis of the iteration can also be indicated e.g. [for every book object].

The sequence diagram for the book renewal use case for the Library Automation Software is shown in fig. The development of the sequence diagram in the development methodology would help us in determining the responsibilities of the different classes; i.e. what methods should be supported by each class

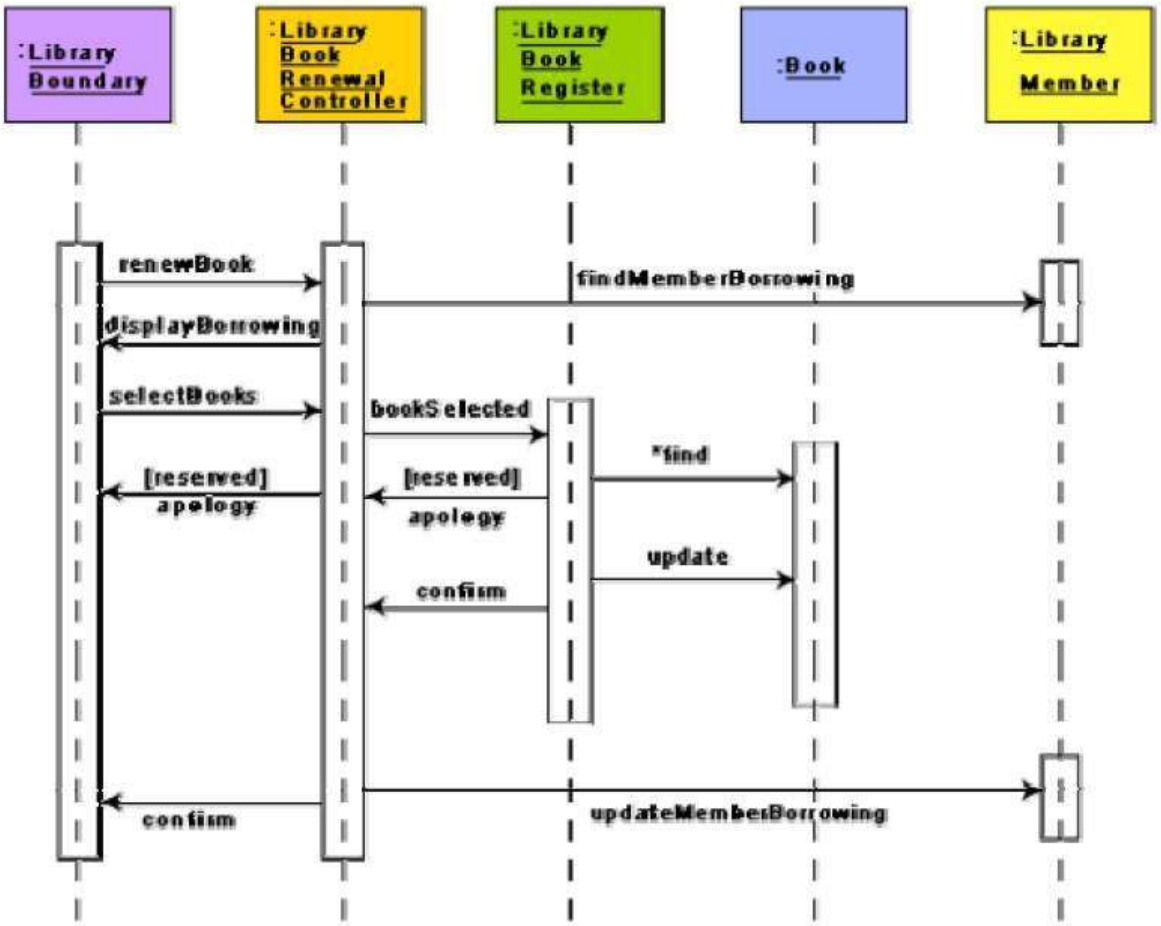


Fig.: Sequence diagram for the renew book use case

Collaboration Diagram

A collaboration diagram shows both structural and behavioral aspects explicitly. This is unlike a sequence diagram which shows only the behavioral aspects. The structural aspect of a collaboration diagram consists of objects and the links existing between them. In this diagram, an object is also called a collaborator. The behavioral aspect is described by the set of messages exchanged among the different collaborators. The link between objects is shown as a solid line and can be used to send messages between two objects. The message is shown as a labeled arrow placed near the link. Messages are prefixed with sequence numbers because they are only way to describe the relative sequencing of the messages in this diagram. The collaboration diagram for the example of fig. 7.13 is shown in fig. 7.14. The use of the collaboration diagrams in our development process would be to help us to determine which classes are associated with which other classes.

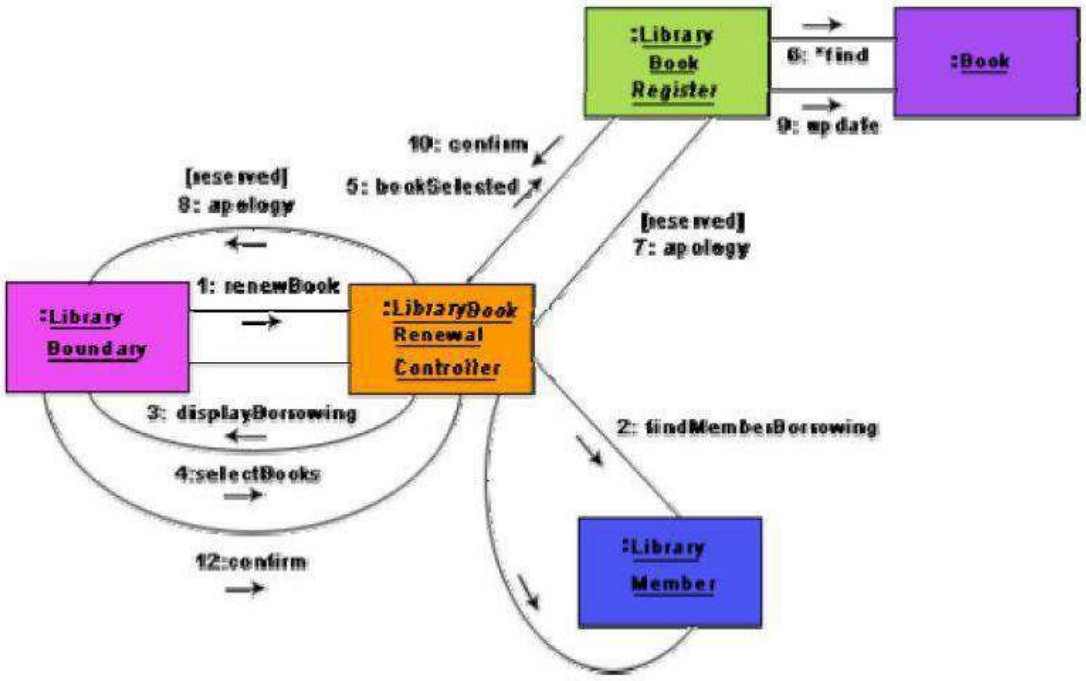


Fig.: Collaboration diagram for the renew book use case s

ACTIVITY DIAGRAMS

The activity diagram is possibly one modeling element which was not present in any of the predecessors of UML. No such diagrams were present either in the works of Booch, Jacobson, or Rumbaugh. It is possibly based on the event diagram of Odell [1992] through the notation is very different from that used by Odell. The activity diagram focuses on representing activities or chunks of processing which may or may not correspond to the methods of classes. An activity is a state with an internal action and one or more outgoing transitions which automatically follow the termination of the internal activity. If an activity has more than one outgoing transitions, then these must be identified through conditions. An interesting feature of the activity diagrams is the swim lanes. Swim lanes enable you to group activities based on who is performing them, e.g. academic department vs. hostel office. Thus swim lanes subdivide activities based on the responsibilities of some components. The activities in a swim lane can be assigned to some model elements, e.g. classes or some component, etc.

Activity diagrams are normally employed in business process modeling. This is carried out during the initial stages of requirements analysis and specification. Activity diagrams can be very useful to understand complex processing activities involving many components. Later these diagrams can be used to develop interaction diagrams which help to allocate activities (responsibilities) to classes.

The student admission process in IIT is shown as an activity diagram in fig. 7.15. This shows the part played by different components of the Institute in the admission procedure. After the fees are received at the account section, parallel activities start at the hostel office, hospital,

and the Department. After all these activities complete (this synchronization is represented as a horizontal line), the identity card can be issued to a student by the Academic section.

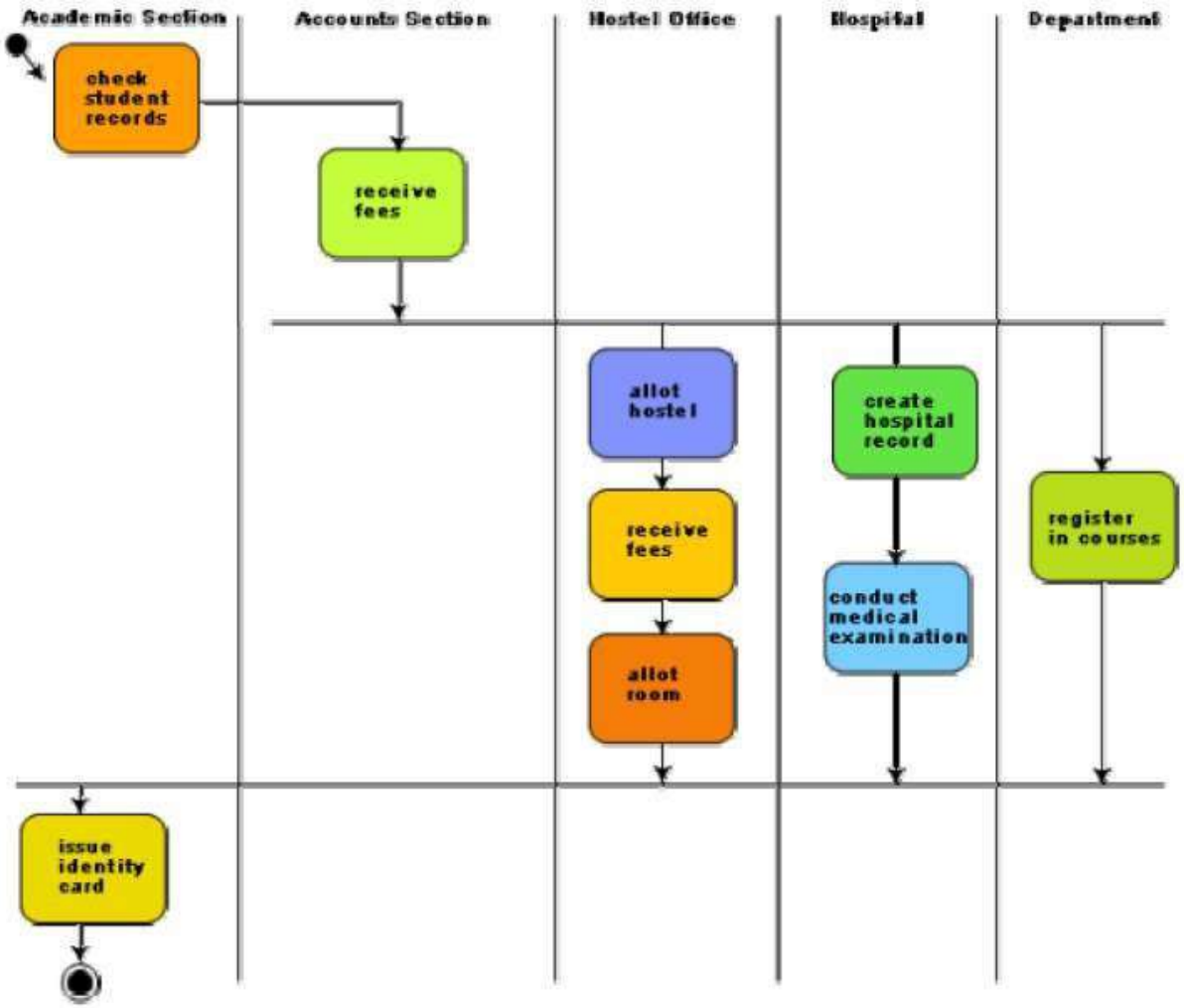


Fig.: Activity diagram for student admission procedure at IIT

Activity diagrams vs. procedural flow charts

Activity diagrams are similar to the procedural flow charts. The difference is that activity diagrams support description of parallel activities and synchronization aspects involved in different activities.

STATE CHART DIAGRAM

A state chart diagram is normally used to model how the state of an object changes in its lifetime. State chart diagrams are good at describing how the behavior of an object changes across several use case executions. However, if we are interested in modeling some behavior that involves several objects collaborating with each other, state chart diagram is not appropriate. State chart diagrams are based on the finite state machine (FSM) formalism.

An FSM consists of a finite number of states corresponding to those of the object being modeled. The object undergoes state changes when specific events occur. The FSM formalism existed long before the object-oriented technology and has been used for a wide variety of applications. Apart from modeling, it has even been used in theoretical computer science as a generator for regular languages.

A major disadvantage of the FSM formalism is the state explosion problem. The number of states becomes too many and the model too complex when used to model practical systems. This problem is overcome in UML by using state charts. The state chart formalism was proposed by David Harel [1990]. A state chart is a hierarchical model of a system and introduces the concept of a composite state (also called nested state).

Actions are associated with transitions and are considered to be processes that occur quickly and are not interruptible. Activities are associated with states and can take longer. An activity can be interrupted by an event.

The basic elements of the state chart diagram are as follows:

- **Initial state.** This is represented as a filled circle.
- **Final state.** This is represented by a filled circle inside a larger circle.
- **State.** These are represented by rectangles with rounded corners.
- **Transition.** A transition is shown as an arrow between two states. Normally, the name of the event which causes the transition is places along side the arrow. A guard to the transition can also be assigned. A guard is a Boolean logic condition. The transition can take place only if the grade evaluates to true. The syntax for the label of the transition is shown in 3 parts: event[guard]/action.

An example state chart for the order object of the Trade House Automation software is shown in fig.

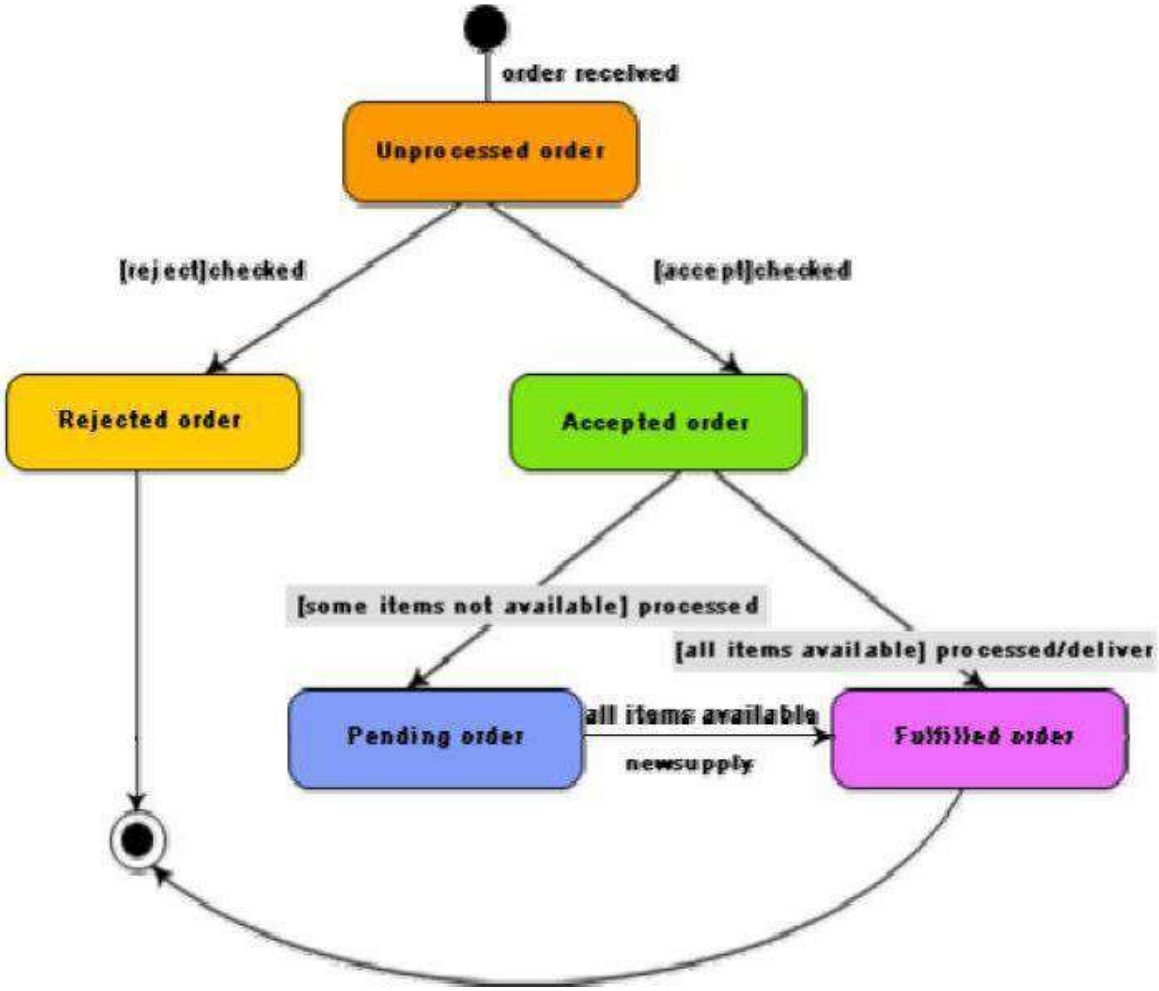


Fig.: State chart diagram for an order object

Activity diagram vs. State chart diagram

• Both activity and state chart diagrams model the dynamic behavior of the system Activity diagram is essentially a flowchart showing flow of control from activity to activity. A state chart diagram shows a state machine emphasizing the flow of control from state to state.

• An activity diagram is a special case of a state chart diagram in which all or most of the states are activity states and all or most of the transitions are triggered by completion of activities in the source state (An activity is an ongoing non-atomic execution within a state machine).

• Activity diagrams may stand alone to visualize, specify, and document the dynamics of a society of objects or they may be used to model the flow of control of an operation. State chart diagrams may be attached to classes, use cases, or entire systems in order to visualize, specify, and document the dynamics of an individual object.

DESIGN PATTERNS

Design patterns are reusable solutions to problems that recur in many applications. A pattern serves as a guide for creating a "good" design. Patterns are based on sound common sense and the application of fundamental design principles. These are created by people who spot repeating themes across designs. The pattern solutions are typically described in terms of class and interaction diagrams. Examples of design patterns are expert pattern, creator pattern, controller pattern etc.

8.1 BASIC PATTERN CONCEPTS:

Design patterns are very useful in creating good software design solutions. In addition to providing the model of a good solution, design patterns include a clear specification of the problem, and also explain the circumstances in which the solution would and would not work. Thus, a design pattern has four important parts:

- The problem.
- The context in which the problem occurs.
- The solution.
- The context within which the solution works.

8.2 TYPES OF PATTERNS

Starting from use in very high-level designs termed as architectural designs, pattern solutions have been defined for use in concrete designs and even in code. Some basic concepts about these three types of patterns as follows:

- **Architectural patterns**

  The architectural patterns identify and provide solutions to problems that are identifiable while carrying out architectural designs.

  Architectural designs concern the overall structure of software systems.

- **Design patterns**

  A design pattern usually suggests a scheme for structuring the classes in a design solution and defines the interaction required among those classes.

  In other words, a design pattern describes some commonly recurring structure of communicating classes that can be used to solve some general design problems.

  Design pattern solutions are typically described in terms of classes, their instances,their roles and collaborations.

- **Idioms**

  Idioms are low-level patterns that are programming language-specific.

  An idiom describes how to implement a solution to a particular problem using the features of a given programming language.

MORE PATTERN CONCEPTS

Few other important pattern concepts in the following.

Patterns versus algorithms

In contrast of algorithms, patterns are more concerned with aspects such as maintainability and ease of development rather than space and time efficiency.

Prons and cons of design patterns

Some advantages of design patterns:

- Design patterns providen a common vocabulary that helps to improve communication among the developers.
- Design patterns help capture and disseminate expert knowledge.
- Use of design patterns help designers to produce designs that are flexible, efficient, and easily maintainable.
- Design patterns guide developers to arrive at correct design decisions and help improve the quality of the design.
- Design patterns reduce the number of design iterations, and help improve the designer productivity.

Antipattern

The following are two types of antipatterns that are popular:

- Those that describe bad solutions to problems which leads to bad situations.
- Those that describe how to avoid bad solutions to problems.

We mention here only a few interesting antipatterns without discussing them in detail,

1. **Input kludge**: This concerns failing to specify and implement a mechanism for handling invalid inputs.
2. **Magic pushbutton:** This concerns coding implementation logic directly within the code of the user interface, rather than performing them in separate classes.
3. **Race hazard:** This concerns failing to see the consequence of all the different orders in which events might take place in practice.

SOME COMMON DESIGN PATTERNS:

Expert Pattern

Problem: Which class should be responsible for doing certain things?

Solution: Assign responsibility to the information expert – the class that has the information necessary to fulfill the required responsibility. The expert pattern expresses the common intuition that objects do things related to the information they have. The class diagram and collaboration diagrams for this solution to the problem of which class should compute the total sales is shown in the fig.

(a)

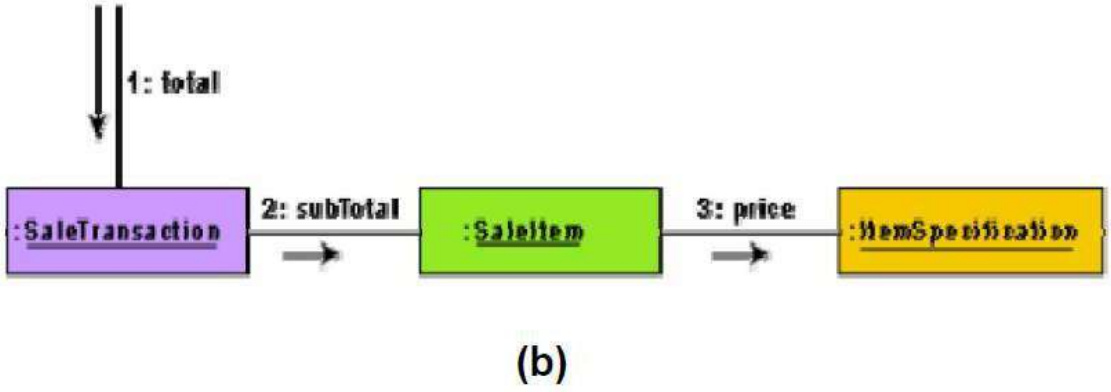


Fig.: Expert pattern: (a) Class diagram (b) Collaboration diagram

Creator Pattern

Problem: Which class should be responsible for creating a new instance of some class?

Solution: Assign a class C1 the responsibility to create an instance of class C2, if one or more of the following are true**:**

- C1 is an aggregation of objects of type C2.
- C1 contains objects of type C2.
- C1 closely uses objects of type C2.
- C1 has the data that would be required to initialize the objects of type C2, when they are created.

Controller Pattern:

Problem: Who should be responsible for handling the actor requests?

Solution: For every use case, there should be a separate controller object which would be responsible for handling requests from the actor. Also, the same controller should be used for all the actor requests pertaining to one use case so that it becomes possible to maintain the necessary information about the state of the use case. The state information maintained by a controller can be used to identify the out-of-sequence actor requests, e.g. whether voucher request is received before arrange payment request.

Façade Pattern:

Problem: How should the services be requested from a service package?

Context in which the problem occurs: A package as already discussed is a cohesive set of classes – the classes have strongly related responsibilities. For example, an RDBMS interface package may contain classes that allow one to perform various operations on the RDBMS.

Solution: A class (such as DBfacade) can be created which provides a common interface to the services of the package.

Model View Separation Pattern:

Problem: How should the non-GUI classes communicate with the GUI classes?
Context in which the problem occurs: This is a very commonly occurring pattern which occurs in almost every problem. Here, model is a synonym for the domain layer objects, view is a synonym for the presentation layer objects such as the GUI objects.
Solution: The model view separation pattern states that model objects should not have direct knowledge (or be directly coupled) to the view objects. This means that there should not be any direct calls from other objects to the GUI objects. This results in a good solution, because the GUI classes are related to a particular application whereas the other classes may be reused.

There are actually two solutions to this problem which work in different circumstances as follows:

Solution 1: Polling or Pull from above

It is the responsibility of a GUI object to ask for the relevant information from the other objects, i.e. the GUI objects pull the necessary information from the other objects whenever required.

This model is frequently used. However, it is inefficient for certain applications. For example, simulation applications which require visualization, the GUI objects would not know when the necessary information becomes available. Other examples are, monitoring applications such as network monitoring, stock market quotes, and so on. In these situations, a "push-from-below" model of display update is required. Since "push-from-below" is not an acceptable solution, an indirect mode of communication from the other objects to the GUI objects is required.

Solution 2: Publish- subscribe pattern

An event notification system is implemented through which the publisher can indirectly notify the subscribers as soon as the necessary information becomes available. An event manager class can be defined which keeps track of the subscribers and the types of events they are interested in. An event is published by the publisher by sending a message to the event manager object. The event manager notifies all registered subscribers usually via a parameterized message (called a callback). Some languages specifically support event manager classes. For example, Java provides the EventListener interface for such purposes.

Intermediary Pattern or Proxy

Problem: How should the client and server objects interact with each other?
Context in the problem occurs: The client and server terms as used here refer to software components existing across a network. The clients are consumers of services provided by the servers.

Solution: A proxy object at the client side can be defined which is a local sit-in for the remote server object. The proxy hides the details of the network transmission. The proxy is responsible for determining the server address, communicating the client request to the server, obtaining the server response and seamlessly passing that to the client. The proxy can also augment (or filter)

information that is exchanged between the client and the server. The proxy could have the same interface as the remote server object so that the client feels as if it is interacting directly with the remote server object and the complexities of network transmissions are abstracted out.

OBJECT ORIENTED ANALYSIS AND DESIGN METHODOLOGY

THE UNIFIED PROCESS

The two main characteristics of the unified process are: use case-driven and iterative. The use case model is the central model. All models that are constructed in the subsequent design activities must conform to the use case model.

The unified process involves iterating over the following four distinct phases as follows.

1.Inception: During this phase, the scope of the project is defined and prototype may be developed to form a clear idea about the project.

2.Elaboration: In this phase, the functional and the non-functional requirements are captured.

3.Construction: During this phase, analysis, design, and implementation activities are carried out. Full text descriptions of use cases are written during the construction phase and each use case is taken up for the start of a new iteration. System features are implemented in a series of short iterations. Each iteration results in an executable release of the software.

4.Transition: During this phase the product is installed in the user's environment and maintained. The design process that we discuss in this section can be undertaken during the construction phase of the unified process.

OVERVIEW OF THE OOAD METHODOLOGY

- The Object-Oriented Analysis and Design(OOAD) methodology that we are going to discuss has shown in Figure
- The use case model is developed first in any user-centric development process such as the unified process, all developed models must conformed to the use case model.
- Therefore, the use case model has a crucial role in the design process, and needs to be developed first.
- The domain model is constructed next through an analysis of the use case model and the SRS document. The domain model is refined into a class diagram through a number of iterations involving the interaction diagrams. Once the class diagram has been constructed, it can be easily be translated to code.
- Throughout the analysis and design process, a glossary is continually and consciously created and maintained.
- A glossary is a dictionary of terms which can help in understanding the various terms(or concepts) used in the constructed model.
- The terms listed in the glossary are essentially concept names.
- The glossary (or model dictionary) lists and all the terms that require explanation in order to improve communication and to reduce the risk of misunderstanding. Maintaining the glossary is an ongoing activity throughout the project.

USE CASE MODEL DEVELOPMENT

An overriding principle while identifying and packaging use cases is that there should be a strong correlation between the GUI prototype, the contents of the users' manual and the use case model of the system.

Each of the menu options in the top-level menu of the GUI would usually correspond to a package in the use case diagram.

Common mistakes committed in use case model development

The following are some common mistakes that beginners commit during use case model development. List of mistakes are:

1. **Clutter:**
2. **Too detailed:**
3. **Omitting text description:**
4. **Overlooking some alternate scenarios:**

DOMAIN METHODOLOGY

Domain modeling is known as conceptual modeling. A domain model is a representation of the concepts or objects appearing in the problem domain. It also captures the obvious relationships among these objects. Examples of such conceptual objects are the Book, BookRegister, MemeberRegister, LibraryMember, etc. The recommended strategy is to quickly create a rough conceptual model where the emphasis is in finding the obvious concepts expressed in the requirements while deferring a detailed investigation. Later during the development process, the conceptual model is incrementally refined and extended.

The objects identified during domain analysis can be classified into three types:

- Boundary objects
- Controller objects
- Entity objects

Boundary objects: The boundary objects are those with which the actors interact. These include screens, menus, forms, dialogs, etc. The boundary objects are mainly responsible for user interaction. Therefore, they normally do not include any processing logic. However, they may be responsible for validating inputs, formatting, outputs, etc. The boundary objects were earlier being called as the interface objects. However, the term interface class is being used for Java, COM/DCOM, and UML with different meaning. A recommendation for the initial identification of the boundary classes is to define one boundary class per actor/use case pair.

Entity objects: These normally hold information such as data tables and files that need to outlive use case execution, e.g. Book, BookRegister, LibraryMember, etc. Many of the entity

objects are “dumb servers”. They are normally responsible for storing data, fetching data, and doing some fundamental kinds of operation that do not change often.

Controller objects: The controller objects coordinate the activities of a set of entity objects and interface with the boundary objects to provide the overall behavior of the system. The responsibilities assigned to a controller object are closely related to the realization of a specific use case. The controller objects effectively decouple the boundary and entity objects from one another making the system tolerant to changes of the user interface and processing logic. The controller objects embody most of the logic involved with the use case realization (this logic may change time to time). A typical interaction of a controller object with boundary and entity objects is shown in fig. Normally, each use case is realized using one controller object.

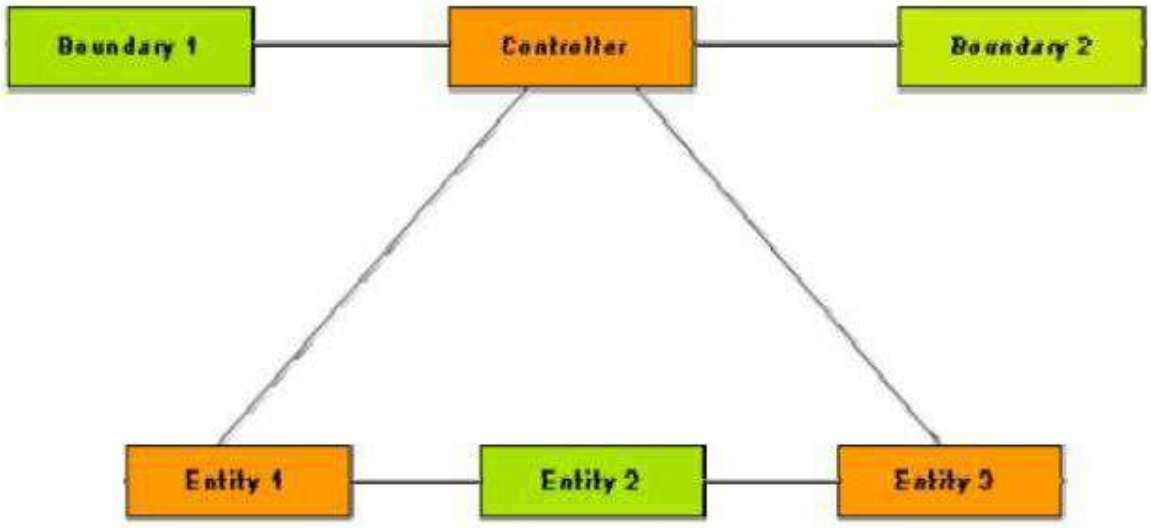


Fig. A typical realization of a use case through the collaboration of boundary, controller, and entity objects

IDENTIFICATION OF ENTITY OBJECTS

One of the most important steps in any object-oriented design methodology is the identification of objects. In fact, the quality of the final design depends to a great extent on the appropriateness of the objects identified. However, to date no formal methodology exists for identification of objects. Several semi-formal and informal approaches have been proposed for object identification. These can be classified into the following broad classes:

- Grammatical analysis of the problem description.
- Derivation from data flow.
- Derivation from the entity relationship (E-R) diagram.

A widely accepted object identification approach is the grammatical analysis approach. Grady Booch originated the grammatical analysis approach [1991]. In Booch’s approach, the

nouns occurring in the extended problem description statement (processing narrative) are mapped to objects and the verbs are mapped to methods. The identification approaches based on derivation from the data flow diagram and the entity-relationship model are still evolving and therefore will not be discussed in this text.

9.6 BOOCH'S OBJECT IDENTIFICATION METHOD

Booch's object identification approach requires a processing narrative of the given problem to be first developed. The processing narrative describes the problem and discusses how it can be solved. The objects are identified by noting down the nouns in the processing narrative. Synonym of a noun must be eliminated. If an object is required to implement a solution, then it is said to be part of the solution space. Otherwise, if an object is necessary only to describe the problem, then it is said to be a part of the problem space. However, several of the nouns may not be objects. An imperative procedure name, i.e., noun form of a verb actually represents an action and should not be considered as an object. A potential object found after lexical analysis is usually considered legitimate, only if it satisfies the following criteria**:**

Retained information. Some information about the object should be remembered for the system to function. If an object does not contain any private data, it can not be expected to play any important role in the system.

Multiple attributes. Usually objects have multiple attributes and support multiple methods. It is very rare to find useful objects which store only a single data element or support only a single method, because an object having only a single data element or method is usually implemented as a part of another object.

Common operations. A set of operations can be defined for potential objects. If these operations apply to all occurrences of the object, then a class can be defined. An attribute or operation defined for a class must apply to each instance of the class. If some of the attributes or operations apply only to some specific instances of the class, then one or more subclasses can be needed for these special objects.

Normally, the actors themselves and the interactions among themselves should be excluded from the entity identification exercise. However, some times there is a need to maintain information about an actor within the system. This is not the same as modeling the actor. These classes are sometimes called surrogates. For example, in the Library Information System (LIS) we would need to store information about each library member. This is independent of the fact that the library member also plays the role of an actor of the system description. Useful abstractions usually result from clever factoring of the problem description into independent and intuitively correct elements.

Although the grammatical approach is simple and intuitively appealing, yet through a naive use of the approach, it is very difficult to achieve high quality results. In particular, it is very difficult to come up with useful abstractions simply by doing grammatical analysis of the problem

INTERACTION MODELING

The primary goal of interaction modeling are the following**:**

- To allocate the responsibility of a use case realization among the boundary, entity, and controller objects. The responsibilities for each class is reflected as an operation to be supported by that class.
- To show the detailed interaction that occur over time among the objects associated with each use case.

CRC cards

The interactions diagrams for only simple use cases that involve collaboration among a limited number of classes can be drawn from an inspection of the use case description. More complex use cases require the use of CRC cards where a number of team members participate to determine the responsibility of the classes involved in the use case realization.

CRC (Class-Responsibility-Collaborator) technology was pioneered by Ward Cunningham and Kent Becka at the research laboratory of Tektronix at Portland, Oregon, USA. CRC cards are index cards that are prepared one per each class. On each of these cards, the responsibility of each class is written briefly. The objects with which this object needs to collaborate its responsibility are also written.

CRC cards are usually developed in small group sessions where people role play being various classes. Each person holds the CRC card of the classes he is playing the role of. The cards are deliberately made small (4 inch ´ 6 inch) so that each class can have only limited number of responsibilities. A responsibility is the high level description of the part that a class needs to play in the realization of a use case. An example CRC card for the BookRegister class of the Library Automation System is shown in fig

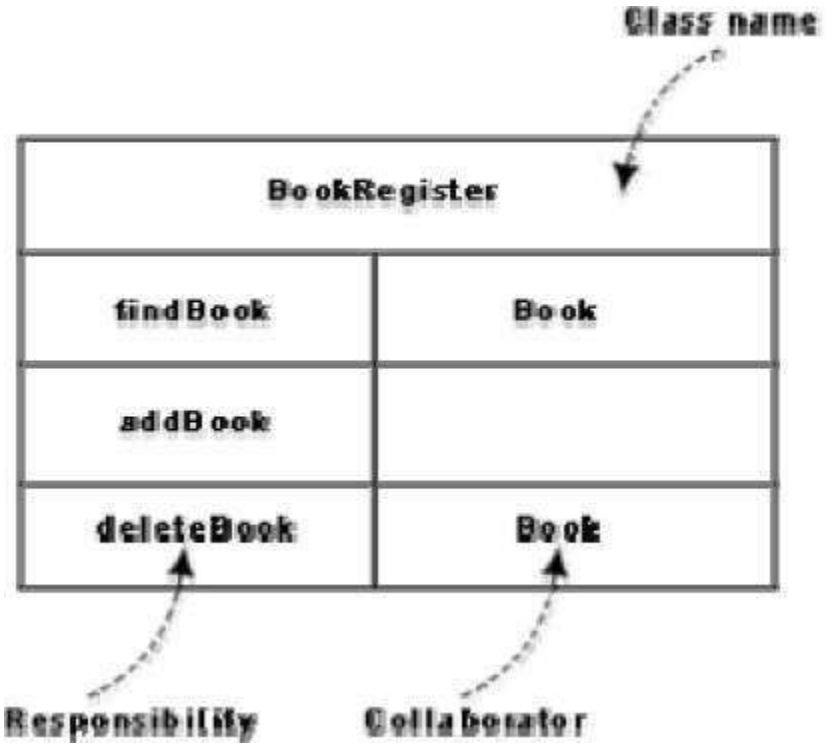


Fig. CRC card for the BookRegister class

OOD GOODNESS CRITERIA

There are several subjective judgments involved in arriving at a good object-oriented design. Therefore, several alternative design solutions to the same problem are possible. In order to be able to determine which of any two designs is better, some criteria for judging the goodness of a design must be identified. The following are some of the accepted criteria for judging the goodness of a design.

1. **Coupling guidelines.** The number of messages between two objects or among a group of objects should be minimum. Excessive coupling between objects is determined to modular design and prevents reuse.
2. **Cohesion guideline.** In OOD, cohesion is aboutthree levels**:**
   - **Cohesiveness of the individual methods.** Cohesiveness of each of the individual method is desirable, since it assumes that each method does only a well-defined function.
   - **Cohesiveness of the data and methods within a class.** This is desirable since it assures that the methods of an object do actions for which the object is naturally responsible, i.e. it assures that no action has been improperly mapped to an object
   - **Cohesiveness of an entire class hierarchy.** Cohesiveness of methods within a class is desirable since it promotes encapsulation of the objects.
3. **Hierarchy and factoring guidelines.** A base class should not have too many subclasses. If too many subclasses are derived from a single base class, then it becomes difficult to understand the design. In fact, there should approximately be no more than 7±2 classes derived from a base class at any level.
4. **Keeping message protocols simple.** Complex message protocols are an indication of excessive coupling among objects. If a message requires more than 3 parameters, then it is an indication of bad design.
5. • **Number of Methods.** Objects with a large number of methods are likely to be more application-specific and also difficult to comprehend – limiting the possibility of their reuse. Therefore, objects should not have too many methods. This is a measure of the complexity of a class. It is likely that the classes having more than about seven methods would have problems.
6. • **Depth of the inheritance tree.** The deeper a class is in the class inheritance hierarchy, the greater is the number of methods it is likely to inherit, making it more complex. Therefore, the height of the inheritance tree should not be very large.
7. • **Number of messages per use case.** If methods of a large number of objects are invoked in a chain action in response to a single message, testing and debugging of the objects becomes complicated. Therefore, a single message should not result in excessive message generation and transmission in a system.

8. • **Response for a class.** This is a measure of the maximum number of methods that an instance of this class would call. If the same method is called more than once, then it is counted only once. A class which calls more than about seven different methods is susceptible to errors.